

OWASP ZAP Security Scan Report

Overview

This document provides a summary of the vulnerabilities and security configurations identified by the **OWASP ZAP (Zed Attack Proxy)** scan on **July 11, 2026**.

The scan predominantly identified **Medium**, **Low**, and **Informational** risk findings. No High-risk vulnerabilities were identified in the automated scan. Below is a categorized breakdown of the alerts found along with standard remediation advice.

General Recommendation: *Most of the findings revolve around missing security headers and improper cookie configurations. Implementing a robust middleware for security headers (like Helmet for Node.js) can resolve many of these issues quickly.*

Findings by Category

1. Security Headers & CSP

These issues indicate that the server is not enforcing client-side security policies, leaving the application more susceptible to Cross-Site Scripting (XSS), Clickjacking, and MIME-sniffing attacks.

- **Content Security Policy (CSP) Header Not Set:** The server does not restrict where resources can be loaded from.
- **CSP: Wildcard Directive:** A CSP is present but uses a wildcard (*), which weakens the policy.
- **CSP: style-src unsafe-inline:** Inline styles are allowed, which can be a vector for CSS injection.
- **Missing Anti-clickjacking Header:** The `X-Frame-Options` header is missing, allowing the site to be framed by malicious domains.
- **X-Content-Type-Options Header Missing:** The server does not prevent the browser from MIME-sniffing the response.
- **Strict-Transport-Security Header Not Set / Disabled:** HSTS is missing, meaning the browser is not forced to communicate over HTTPS.

2. Cookie Security

Cookies are being set without the proper flags to protect them from unauthorized access or cross-site request forgery (CSRF).

- **Cookie No HttpOnly Flag:** Cookies can be accessed via client-side scripts (e.g., JavaScript), making them vulnerable to XSS.
- **Cookie Without Secure Flag:** Cookies can be transmitted over unencrypted HTTP connections.
- **Cookie without SameSite Attribute:** The `SameSite` attribute is missing, making the application potentially vulnerable to CSRF.
- **Cookie with SameSite Attribute None:** `SameSite=None` is explicitly set, allowing cross-site cookie sending (must be paired with the `Secure` flag).
- **Loosely Scoped Cookie:** Cookies are scoped too broadly, potentially exposing them to other subdomains.

3. Information Disclosure

The application might be leaking sensitive information that could be used by an attacker to map the application or exploit it.

- **Information Disclosure - JWT in Browser localStorage:** JSON Web Tokens (JWT) are being stored in `localStorage`, which is accessible to any JavaScript on the page (high risk if XSS is present).
- **Information Disclosure - Information in Browser localStorage:** General sensitive information is being stored in the browser's local storage.
- **Information Disclosure - Sensitive Information in URL:** Sensitive data (like tokens or IDs) might be passed in the URL parameters.
- **Private IP Disclosure:** Internal IP addresses are being leaked in the HTTP responses.
- **Timestamp Disclosure - Unix:** Unix timestamps are present, which could help an attacker determine server uptime or predict random number generation.
- **Information Disclosure - Suspicious Comments:** Source code contains comments that might reveal internal architecture or developer notes.

4. Cross-Domain & Cache Misconfigurations

- **Cross-Domain Misconfiguration:** CORS policies or cross-domain policies might be overly permissive (e.g., `Access-Control-Allow-Origin: *`).
- **Re-examine Cache-control Directives:** The `Cache-Control` header might be allowing sensitive pages to be cached by the browser or intermediate proxies.
- **Retrieved from Cache:** Some responses were served from the cache, which might indicate caching of dynamic content.

5. Application Flow & Informational

- **Authentication Request Identified:** ZAP identified where authentication happens.
 - **Session Management Response Identified:** ZAP identified how sessions are managed.
 - **Modern Web Application:** The application uses modern web technologies (like SPAs) which ZAP flags for informational purposes.
-

Action Plan / Next Steps

1. **Secure the Cookies:** Update your server or application code to ensure all session and sensitive cookies are set with `HttpOnly`, `Secure`, and `SameSite=Lax` (or `Strict`).
2. **Implement Security Headers:** Add strict security headers to all HTTP responses:
 - `Strict-Transport-Security: max-age=31536000; includeSubDomains`
 - `X-Content-Type-Options: nosniff`
 - `X-Frame-Options: DENY` (or `SAMEORIGIN`)
 - Reconfigure your **Content Security Policy (CSP)** to remove wildcards (*) and `unsafe-inline`.
3. **Review Local Storage:** Avoid storing sensitive tokens (like JWTs) in `localStorage`. Consider using `HttpOnly` cookies for session tokens instead.
4. **Remove Information Leaks:** Review URL parameters to ensure no sensitive data is passed, and strip private IPs or internal comments from production builds.